

0.33.1. Écrire soi-même sa fonction — de base, ailleurs

La fonction du 0.33, appelée pour le B sur la ligne 8.

1. Ce qu'il faut comprendre

C'est le 0.33, avec une autre lettre, le **B** (`ALPHABET[1]`, la 2e lettre), et une autre place : la **ligne 8** dans les deux appels.

La fonction ne change pas : c'est tout l'intérêt d'une fonction. On lui donne d'autres renseignements (la ligne, la tuile), elle fait le même travail ailleurs.

C'est la version de base : une seule lettre, à la deuxième place. Le 0.33.2 met les deux ensemble.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1606 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 0;

// Notre fonction, la même qu'au 0.33 (voir ses commentaires là-bas).
```

2. UNE FONCTION — MON_DEPLACE_X()

```
uint8_t mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas) {
    uint8_t recule = pas > 127;
    if (recule) pas = 0 - pas;
    poser(colonne, ligne, tuile);
    while (pas > 0) {
        for (uint8_t i = 0; i < 15; i++) image();
        if (recule && colonne == 0) break;
        if (!recule && colonne == 19) break;
        effacer(colonne, ligne, 1);
        if (recule) colonne--;
        else colonne++;
        poser(colonne, ligne, tuile);
        pas--;
    }
    return colonne;
}
```

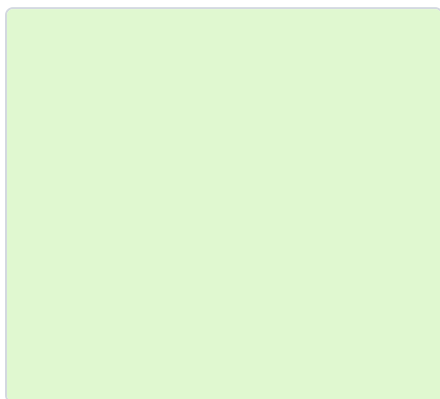
3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    x = mon_deplace_x(x, 8, ALPHABET[1], 5);    // le B, ligne 8 : x vaut 5
    x = mon_deplace_x(x, 8, ALPHABET[1], -5);   // retour : x vaut 0

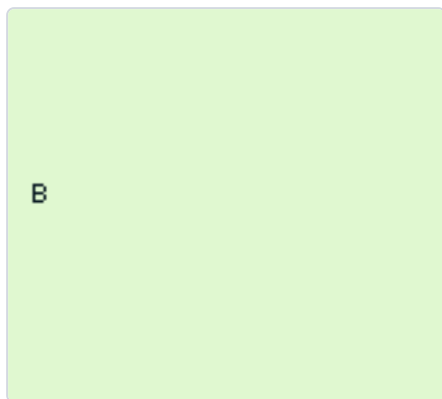
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

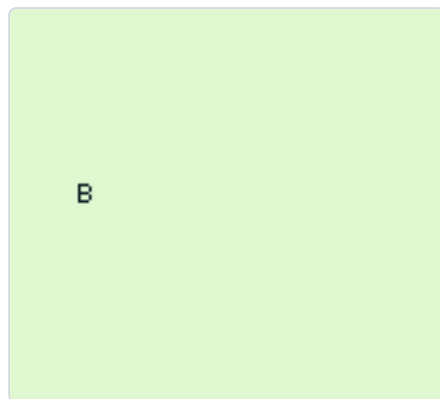
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un B qui va et revient sur la ligne 8, grâce à notre fonction.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le B passe par (5, 8)
- ✓ le B finit en (0, 8) (0,8)

Ensuite : **0.33.2. Écrire soi-même sa fonction — doublé, deux positions** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++